Experiment 'p_6_7_reverse_array_seq' Results

June 4, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_6_7_reverse_array_seq
Natural language statement: Write a method that reverses the sequence of elements in an array. For example, if you call the method with the array [1, 4, 9, 16, 9, 7, 4, 9, 11] then the array is changed to [11, 9, 4, 7, 9, 16, 9, 4, 1].
Method signature: p_6_7_reverse_array_seq(inputs: seq<int>) returns (reversed: seq<int>)

Ensures

- `|reversed| == |inputs|`
- `forall i :: 0 <= i < |reversed| ==> reversed[i] == inputs[|inputs| - 1 - i]`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method that reverses the sequence of elements in an array. For example, if you
    ↪ call the method with the array [1, 4, 9, 16, 9, 7, 4, 9, 11] then the array is
    ↪ changed to [11, 9, 4, 7, 9, 16, 9, 4, 1].

The signature should be:

p_6_7_reverse_array_seq(inputs: seq<int>) returns (reversed: seq<int>)

The method should respect the following contract:

ensures |reversed| == |inputs|, ensures forall i :: 0 <= i < |reversed| ==> reversed[i]
    ↪ == inputs[|inputs| - 1 - i]

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-Dn1ep3pIjtjvwRwoF1JnkV1njGBhu
Model: gpt-5.5-2026-04-23
Created at: 1780576815
Finish reason: stop
Usage: 470 tokens in, and 7188 tokens out

```
{
  reversed := [];
  var i := |inputs|;
  while i > 0
    invariant 0 <= i <= |inputs|
    invariant |reversed| == |inputs| - i
    invariant forall j :: 0 <= j < |reversed| ==> reversed[j] == inputs[|inputs| - 1 - j]
    decreases i
  {
    var oldReversed := reversed;
    var oldI := i;

    forall j | 0 <= j < |oldReversed|
      ensures oldReversed[j] == inputs[|inputs| - 1 - j]
    {
      assert oldReversed == reversed;
      assert |oldReversed| == |reversed|;
      assert 0 <= j < |reversed|;
```

```
    assert oldReversed[j] == reversed[j];
    assert reversed[j] == inputs[|inputs| - 1 - j];
  }

  i := i - 1;
  assert oldI > 0;
  assert i == oldI - 1;
  assert 0 <= i < |inputs|;

  reversed := oldReversed + [inputs[i]];
  assert |oldReversed| == |inputs| - oldI;
  assert |reversed| == |oldReversed| + 1;
  assert |reversed| == |inputs| - i;

  forall j | 0 <= j < |reversed|
    ensures reversed[j] == inputs[|inputs| - 1 - j]
  {
    if j < |oldReversed| {
      assert 0 <= j < |oldReversed|;
      assert reversed[j] == (oldReversed + [inputs[i]])[j];
      assert (oldReversed + [inputs[i]])[j] == oldReversed[j];
      assert oldReversed[j] == inputs[|inputs| - 1 - j];
    } else {
      assert j >= |oldReversed|;
      assert j < |oldReversed| + 1;
      assert j == |oldReversed|;
      assert |[inputs[i]]| == 1;
      assert j - |oldReversed| == 0;
      assert 0 <= j - |oldReversed| < |[inputs[i]]|;
      assert reversed[j] == (oldReversed + [inputs[i]])[j];
      assert (oldReversed + [inputs[i]])[j] == [inputs[i]][j - |oldReversed|];
      assert [inputs[i]][0] == inputs[i];
      assert reversed[j] == inputs[i];
      assert |inputs| - 1 - j == i;
      assert reversed[j] == inputs[|inputs| - 1 - j];
    }
  }
 }

 assert i == 0;
 assert |reversed| == |inputs|;
 assert forall j :: 0 <= j < |reversed| ==> reversed[j] == inputs[|inputs| - 1 - j];
}
```

Final Program

```
method p_6_7_reverse_array_seq(inputs: seq<int>) returns (reversed: seq<int>)
      ensures |reversed| == |inputs|
      ensures forall i :: 0 <= i < |reversed| ==> reversed[i] == inputs[|inputs| - 1 - i
          ↪ ]
```

```
{
  reversed := [];
  var i := |inputs|;
  while i > 0
    invariant 0 <= i <= |inputs|
    invariant |reversed| == |inputs| - i
    invariant forall j :: 0 <= j < |reversed| ==> reversed[j] == inputs[|inputs| - 1 - j]
    decreases i
  {
    var oldReversed := reversed;
    var oldI := i;

    forall j | 0 <= j < |oldReversed|
      ensures oldReversed[j] == inputs[|inputs| - 1 - j]
    {
      assert oldReversed == reversed;
      assert |oldReversed| == |reversed|;
      assert 0 <= j < |reversed|;
      assert oldReversed[j] == reversed[j];
      assert reversed[j] == inputs[|inputs| - 1 - j];
    }

    i := i - 1;
    assert oldI > 0;
    assert i == oldI - 1;
    assert 0 <= i < |inputs|;

    reversed := oldReversed + [inputs[i]];
    assert |oldReversed| == |inputs| - oldI;
    assert |reversed| == |oldReversed| + 1;
    assert |reversed| == |inputs| - i;

    forall j | 0 <= j < |reversed|
      ensures reversed[j] == inputs[|inputs| - 1 - j]
    {
      if j < |oldReversed| {
        assert 0 <= j < |oldReversed|;
        assert reversed[j] == (oldReversed + [inputs[i]])[j];
        assert (oldReversed + [inputs[i]])[j] == oldReversed[j];
        assert oldReversed[j] == inputs[|inputs| - 1 - j];
      } else {
        assert j >= |oldReversed|;
        assert j < |oldReversed| + 1;
        assert j == |oldReversed|;
        assert |[inputs[i]]| == 1;
        assert j - |oldReversed| == 0;
        assert 0 <= j - |oldReversed| < |[inputs[i]]|;
        assert reversed[j] == (oldReversed + [inputs[i]])[j];
        assert (oldReversed + [inputs[i]])[j] == [inputs[i]][j - |oldReversed|];
        assert [inputs[i]][0] == inputs[i];
        assert reversed[j] == inputs[i];
```

```
      assert |inputs| - 1 - j == i;
      assert reversed[j] == inputs[|inputs| - 1 - j];
    }
   }
  }

  assert i == 0;
  assert |reversed| == |inputs|;
  assert forall j :: 0 <= j < |reversed| ==> reversed[j] == inputs[|inputs| - 1 - j];
}
```

Total Token Usage

Input tokens: 470
Output tokens: 7188
Reasoning tokens: 6399
Sum of 'total tokens': 7658

Experiment Timings

Overall Experiment started at 1780576814418, ended at 1780576969439, lasting 155021ms (155.02 seconds)
Iteration #1 started at 1780576814419, ended at 1780576969439, lasting 155020ms (155.02 seconds)